MODULE-2

INTERNETWORK QUALITY OF SERVICE

2.1.QoS Architectural Frame Work

This Recommendation provides an architectural framework for support of Quality of Service (QoS) in packet networks. Central to the architectural framework is a set of generic network mechanisms (or QoS building blocks) for controlling the network service response to a service request, which can be specific to a network element, or for signalling between network elements, or for controlling and administering traffic across a network. Distributed across three logical planes (namely the Control Plane, Data Plane and Management Plane), the building blocks can be used in combination to form various approaches for delivering the satisfactory collective effect of varying service performance required by a range of applications, such as file transfer and multimedia conferencing.

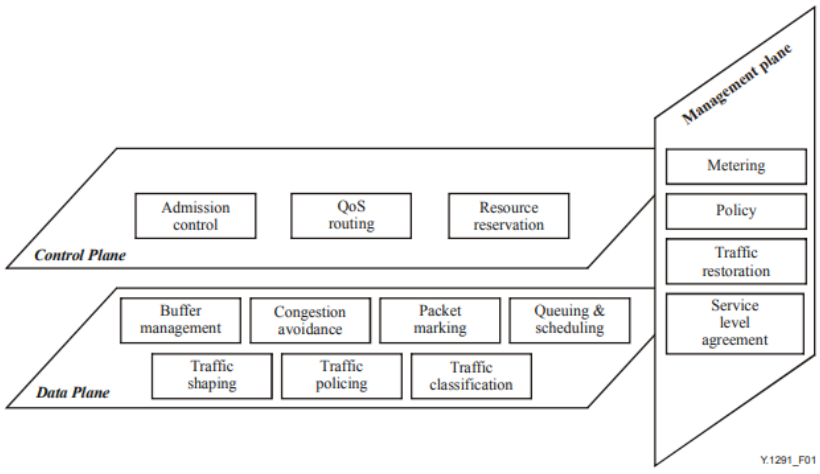


Fig 1: Architectural framework for QoS support

The QoS architectural framework is a set of generic network mechanisms for controlling the network service response to a service request, which can be specific to a network element, or for signalling between network elements, or for controlling and administering traffic across a network. (Please note that the building blocks should not be deemed end to end.) As depicted in Figure 1, the building blocks are organized into three planes:

- Control Plane, which contains mechanisms dealing with the pathways through which user traffic travels. These mechanisms include admission control, QoS routing, and resource reservation.
- Data Plane, which contains mechanisms dealing with the user traffic directly. These mechanisms include buffer management, congestion avoidance, packet marking, queuing and scheduling, traffic classification, traffic policing and traffic shaping.
- Management Plane, which contains mechanisms dealing with the operation, administration, management aspects of the network. These mechanisms include Service Level Agreement (SLA), traffic restoration, metering and recording, and policy.

A QoS building block may be specific to a network node (as exemplified by buffer management) or applicable to a network segment (as exemplified by QoS routing). The latter, in particular, requires signalling between network nodes, whether they are part of a network segment that is end to end, end to edge, edge to edge, or network to network. Signalling can take place in any of the three logical planes. When taking place in the Control or Management Plane, signalling entails the use of a signalling protocol. Because of its unique properties, this Recommendation treats signalling as part of interactions among QoS building blocks and discusses it in the corresponding clause. It is important to note that the QoS architectural framework is logical and puts no constraint on how a building block is realized. As such, the implementation of a building block can be, for instance, distributed or centralized. The following clauses further describe the building blocks according to the planes.

2.1.1.QoS routing

In its narrow definition, QoS routing concerns the selection of a path satisfying the QoS requirements of a flow. The path selected is most likely not the traditional shortest path. Depending on the specifics and the number of QoS metrics involved, computation required for path selection can become prohibitively expensive as the network size grows. Hence practical QoS routing schemes consider mainly cases for a single QoS metric (e.g., bandwidth or delay) or for dual QoS metrics (e.g., cost-delay, cost-bandwidth, and bandwidth-delay)1. To further reduce the complexity of path

computation, various routing strategies exist. According to how the state information is maintained and how the search of feasible paths is carried out, there are strategies such as source routing, distributed routing, and hierarchical routing [Chen]. In addition, according to how multiple QoS metrics are handled, there are strategies such as metric ordering and sequential filtering, which may trade global optimality with reduced computational complexity The path selection process involves the knowledge of the flow's QoS requirements and characteristics and (frequently changing) information on the availability of network resources (expressed in terms of standard metrics such as available bandwidth and delay). The knowledge is typically obtained and distributed with the aid of signalling protocols. For example, RSVP [IETF RFC 2205] can be used for conveying the flow's requirements and characteristics and OSPF extensions as defined in IETF RFC 2676 for resource availability. Compared with shortest-path routing that selects optimal routes based on a relatively constant metric (i.e., hop count or cost), QoS routing tends to entail more frequent and complex path computation and more signalling traffic [Apostolopoulos]. It is important to note that QoS routing provides a means to determine only a path that can likely accommodate the requested performance. To guarantee performance on a selected path, QoS routing needs to be used in conjunction with resource reservation to reserve necessary network resources along the path. QoS routing can also be generalized to apply to traffic engineering. (Concerning slowly-changing traffic patterns over a long time-scale and a coarse granularity of traffic flows, traffic engineering encompasses traffic management, capacity management, traffic measurement and modelling, network modelling, and performance analysis.) To this end, routing selection often takes into account a variety of constraints such as traffic attributes, network constraints, and policy constraints [IETF RFC 3272]. Such generalized QoS routing is also called constraint-based routing, which can afford path selection to bypass congested spots (or to share load) and improve the overall network utilization as well as automate enforcement of traffic engineering policies.

2.2.Integerated Service Architecture

An Overview of Int-Serv

IntServ was defined in IETF RFC 1633, which proposed the resource reservation protocol (RSVP) as a working protocol for signaling in the IntServ architecture. This protocol assumes that resources are reserved for every flow requiring QoS at every router hop in the path between receiver and transmitter using end-to-end signaling.
The IntServ model for IP QoS architecture defines three classes of service based on applications'delay requirements (from highest performance to lowest):

· Guaranteed-service class - with bandwidth, bounded delay, and no-loss guarantees;
· Controlled-load service class - approximating best-effort service in a lightly loaded network, which provides for a form of statistical delay service agreement (nominal delay) that will not be violated more often than in an unloaded network;
· Best-effort service class - similar to that which the Internet currently offers, which is further partitioned into three categories:

· interactive burst (e.g., Web),
· interactive bulk (e.g., FTP) and
· asynchronous (e.g., e-mail)

The main point is that the guaranteed service and controlled load classes are based on quantitative service requirements, and both require signaling and admission control in network nodes. These services can be provided either per-flow or per-flow-aggregate, depending on flow concentration at different points in the network. Although the IntServ architecture need not be tied to any particular signaling protocol, Resource Reservation Protocol (RSVP) described below, is often regarded as the signaling protocol in IntServ. Best-effort service, on the other hand, does not require signaling.

2.2.1 Integrated Services Digital Network (ISDN)

These are a set of communication standards for simultaneous digital transmission of voice, video, data, and other network services over the traditional circuits of the public

switched telephone network. Before *Integrated Services Digital Network (ISDN)*, the telephone system was seen as a way to transmit voice, with some special services available for data. The main feature of ISDN is that it can integrate speech and data on the same lines, which were not available in the classic telephone system.

ISDN is a circuit-switched telephone network system, but it also provides access to packet-switched networks that allows digital transmission of voice and data. This results in potentially better voice or data quality than an analog phone can provide. It provides a packet-switched connection for data in increments of 64 kilobit/s. It provided a maximum of 128 kbit/s bandwidth in both upstream and downstream directions. A greater data rate was achieved through channel bonding. Generally, ISDN B-channels of three or four BRIs (six to eight 64 kbit/s channels) are bonded.

In the context of the OSI model, ISDN is employed as the network in data-link and physical layers but commonly ISDN is often limited to usage to Q.931 and related protocols. These protocols introduced in 1986 are a set of signaling protocols establishing and breaking circuit-switched connections, and for advanced calling features for the user. ISDN provides simultaneous voice, video, and text transmission between individual desktop videoconferencing systems and group videoconferencing systems.

ISDN Interfaces:

The following are the interfaces of ISDN:

1. **Basic Rate Interface (BRI)**

There are two data-bearing channels (‘B’ channels) and one signaling channel (‘D’ channel) in BRI to initiate connections. The B channels operate at a maximum of 64 Kbps while the D channel operates at a maximum of 16 Kbps. The two channels are independent of each other. For example, one channel is used as a TCP/IP connection to a location while the other channel is used to send a fax to a remote location. In iSeries ISDN supports a basic rate interface (BRl).

The basic rate interface (BRl) specifies a digital pipe consisting of two B channels of 64 Kbps each and one D channel of 16 Kbps. This equals a speed of 144 Kbps. In addition, the BRl service itself requires an operating overhead of 48 Kbps. Therefore a digital pipe of 192 Kbps is required.

2. **Primary Rate Interface (PRI)**

Primary Rate Interface service consists of a D channel and either 23 or 30 B channels depending on the country you are in. PRI is not supported on the iSeries. A digital pipe with 23 B channels and one 64 Kbps D channel is present in the usual Primary Rate Interface (PRI). Twenty-three B channels of 64 Kbps each and one D channel of 64 Kbps equals 1.536 Mbps. The PRI service uses 8 Kbps of overhead also. Therefore PRI requires a digital pipe of 1.544 Mbps.

3. **Broadband-ISDN (B-ISDN)**

Narrowband ISDN has been designed to operate over the current communications infrastructure, which is heavily dependent on the copper cable however B-ISDN relies mainly on the evolution of fiber optics. According to CCITT B-ISDN is best described as 'a service requiring transmission channels capable of supporting rates greater than the primary rate.

ISDN Services:

ISDN provides a fully integrated digital service to users. These services fall into 3 categories- bearer services, teleservices, and supplementary services.

1. **Bearer Services**

Transfer of information (voice, data, and video) between users without the network manipulating the content of that information is provided by the bearer network. There is no need for the network to process the information and therefore does not change the content. Bearer services belong to the first three layers of the OSI model. They are well defined in the ISDN standard. They can be provided using circuit-switched, packet-switched, frame-switched, or cell-switched networks.

2. **Teleservices**

   In this, the network may change or process the contents of the data. These services correspond to layers 4-7 of the OSI model. Teleservices rely on the facilities of the bearer services and are designed to accommodate complex user needs. The user need not be aware of the details of the process. Teleservices include telephony, teletex, telefax, videotex, telex, and teleconferencing. Though the ISDN defines these services by name yet they have not yet become standards.

3. **Supplementary Service**

   Additional functionality to the bearer services and teleservices are provided by supplementary services. Reverse charging, call waiting, and message handling are examples of supplementary services which are all familiar with today's telephone company services.

Principle of ISDN:

The ISDN works based on the standards defined by ITU-T (formerly CCITT). The Telecommunication Standardization Sector (ITU-T) coordinates standards for telecommunications on behalf of the International Telecommunication Union (ITU) and is based in Geneva, Switzerland. The various principles of ISDN as per ITU-T recommendation are:

- To support switched and non-switched applications
- To support voice and non-voice applications
- Reliance on 64-kbps connections
- Intelligence in the network
- Layered protocol architecture
- Variety of configurations

Advantages of ISDN:

- ISDN channels have a reliable connection.
- ISDN is used to facilitate the user with multiple digital channels.
- It has faster data transfer rate.
- Efficient use of bandwidth
- Improved call quality
- Greater flexibility
- Integrated services

Disadvantages of ISDN:

- ISDN lines costlier than the other telephone system.
- It requires specialized digital devices.
- It is less flexible.
- Limited coverage
- High installation and maintenance costs
- Obsolescence
- Limited features

2.3. RSVP - Differentiated Services

Using a method similar to the switched virtual circuit (SVC) of asynchronous transfer mode (ATM) networks, IntServ uses RSVP between senders and receivers for per-flow signaling (Fig.5). RSVP messages traverse the network to request and reserve resources. Routers along the path, including core routers, must maintain soft states for RSVP flows. (A soft state is a temporary state governed by the periodic expiration of resource reservations, so that no explicit path tear down request is required. Soft states are refreshed by periodic RSVP messages.)

RSVP is a set-up protocol providing a receiver-based, guaranteed, end-to-end QoS pipe. A reserved pipe is created in the following manner: First, PATH messages flow from the sender downstream to discover the data path. An RESV message, originating from a receiver and traveling in the reverse direction of the PATH messages, attempts to set local Integrated Services standard (IntServ) reservation for the flow. Each node along the path may either admit or reject the reservation subject to capacity or policy admission controls.

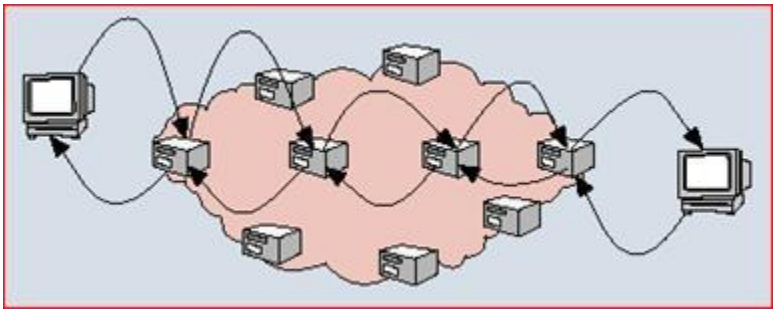


Figure 5 - Classical operation of RSVP.

Advantages of IntServ

The major advantage of IntServ is that it provides service classes, which closely match the different application types described earlier and their requirements. For example, the guaranteed service class is particularly well suited to the support of critical, intolerant applications. On the other hand, critical, tolerant applications and some adaptive applications can generally be efficiently supported by controlled load services. Other adaptive and elastic applications are accommodated in the best-effort service class.

A major characteristic of IntServ is that it leaves the existing best-effort service class mostly unchanged (except for a further subdivision of the class), so it does not involve any change to existing applications. This is an important property since IntServ is then capable of providing this class of service as efficiently as the current Internet. IntServ also leaves the forwarding mechanism in the network unchanged. This allows for an incremental deployment of the architecture, while allowing end systems that have not been upgraded to support IntServ to be able to receive data from any IntServ class (with, of course, a possible loss of guarantee).

IntServ provides a very interesting set of service classes that, although maybe not ideal, represent an excellent approximation of the kind of services required in a global telecommunication platform since it does not discriminate against any applications.

Disadvantages of IntServ

Although IntServ is a straightforward QoS model, end-to-end service guarantees cannot be supported unless all nodes along the route support IntServ. This is obviously so because any best-effort node along any route can treat packets in such a way that the end-to-end service agreements are violated. In the case of end-end implementation of IntServ QoS model, it is recognized by the industry that the support of per-flow guarantees in the core of the Internet will pose severe scalability problems. Therefore, scalability is a key architectural concern for IntServ, since it requires end-to-end signaling and must maintain a per-flow soft state at every router along the path. Other concerns are, how to authorize and prioritize reservation requests, and what happens when signaling is not deployed end-to-end. Because of these issues, it is generally accepted that IntServ is a better candidate for enterprise networks (i.e., for access networks), where user flows can be managed at the desktop user level, than for large service provider backbones. A hybrid model (RSVP-DS) that uses RSVP at the edges and DiffServ in the backbone has been proposed and seems to be winning consensus as a backbone service architecture concept.

Finally, although the sub classing of best-effort service, although already a significant improvement on the flat best-effort service currently provided in the Internet, finer-

grained subclassing of the best-effort service class may be desirable in a commercial network.

2.3.1.Differentiated Services

Differentiated Services Architecture (Diff-Serv)

Diff-Serv is the product of an IETF working group that has defined a more scalable way to apply IP QoS. It has particular relevance to service provider networks. *Diff-Serv minimizes signaling and concentrates on aggregated flows and per hop behaviour (PHB) applied to a network-wide set of traffic classes.* Arriving flows are classified according to pre-determined rules, which aggregate many application flows into a limited and manageable set (perhaps 2 to 8) of class flows.

Traffic entering the network domain at the edge router is first classified for consistent treatment at each transit router inside the network. Treatment will usually be applied by separating traffic into different queues according to the class of traffic, so that high-priority packets can be assigned the appropriate priority level at an output port.

DiffServ approach separates the classification and queuing functions. Packets carry self-evident priority marking in the Type-of-Service byte inside packet headers. (ToS byte is part of the legacy IP architecture.) IP Precedence (IPP) defines eight priority levels. DiffServ, its emerging replacement, reclaims the entire ToS byte to define up to a total of 256 levels. The priority value is interpreted as an index into a Per-Hop Behavior (PHB) that defines the way a single network node should treat a packet marked with this value, so that it will provide consistent multi-hop service. In many cases, PHBs are implemented using some of the queuing disciplines mentioned earlier. This method allows for an efficient index classification that is considered to be highly scalable and best suited for backbone use. However, translating PHBs into end-to-end QoS is not a trivial task. Moreover, inter-domain environments may require a concept known as “bandwidth brokerage,” which is still in the early research stage.

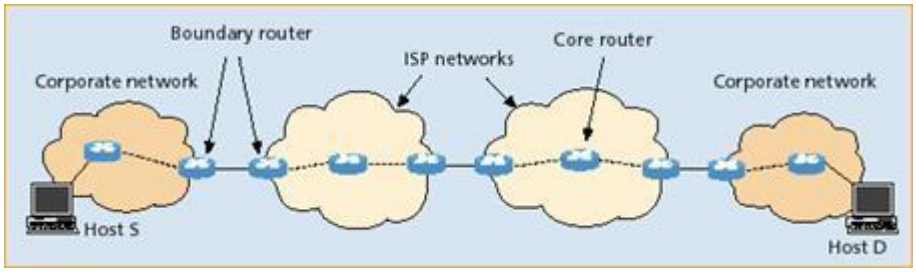


Figure 6 - End-to-end transport from host S to host D under the DiffServ architecture

DiffServ outlines an initial architectural philosophy that serves as a framework for inter-provider agreements and makes it possible to extend QoS beyond a single network domain. *The DiffServ framework is more scalable than IntServ because it handles flow aggregates and minimizes signaling, thus avoiding the complexity of per-flow soft states at each node. Diff-Serv will likely be applied most commonly in enterprise backbones and in service provider networks*.

There will be domains where IntServ and DiffServ coexist, so there is a need to interwork them at boundaries. This interworking will require a set of rules governing the aggregation of individual flows into class flows suitable for transport through a Diff-Serv domain. Several draft interworking schemes have been submitted to the IETF.

An Overview of DiffServ

To provide QoS support, a network must somehow allow for controlled unfairness in the use of its resources. Controlling to a granularity as fine as a flow of data requires advanced signaling protocols. By recognizing that most of the data flows generated by different applications can be ultimately classified into a few general categories (i.e., traffic classes), the DiffServ architecture aims at providing simple and scalable service differentiation. It does this by discriminating and treating the data flows according to their traffic class, thus providing a logical separation of the traffic in the different classes.

In DiffServ, scalability and flexibility are achieved by following a hierarchical model for network resource management:

· **Interdomain resource management**:

Unidirectional service levels, and hence traffic contracts, are agreed at each boundary point between a customer and a provider for the traffic entering the provider network.

· **Intradomain resource management:**

The service provider is solely responsible for the configuration and provisioning of resources within its domain (i.e., the network). Furthermore, service policies are also left to the provider.

At their boundaries, service providers build their offered services with a combination of traffic classes (to provide controlled unfairness), traffic conditioning (a function that modifies traffic characteristics to make it conform to a traffic profile and thus ensure that traffic contracts are respected), and billing (to control and balance service demand). Provisioning and partitioning of both boundary and interior resources are the responsibility of the service provider and, as such, outside the scope of DiffServ. For example, DiffServ does not impose either the number of traffic classes or their characteristics on a service provider.

Although traffic classes are nominally supported by interior routers, DiffServ does not impose any requirement on interior resources and functionalities. For example, traffic conditioning (i.e., metering, marking, shaping, or dropping) in the interior of a network is left to the discretion of the service providers.

If each packet conveyed across a service provider's network simply carries in its header an identification of the traffic class (called a DS codepoint) to which it belongs, the network can easily provide a different level of service to each class. It does this by appropriately treating the corresponding packets, say, by selecting the appropriate per-hop behavior (PHB) for each packet. In both IPv4 and IPv6, the traffic class is denoted by use of the DS header field.

It must be noted that DiffServ is based on local service agreements at customer/provider boundaries. Therefore, end-to-end services will be built by concatenating such local agreements at each domain boundary along the route to the final destination. The concatenation of local services to provide meaningful end-to-end services is still an open research issue.

The net result of the DiffServ approach is that per-flow state is avoided within the network, since individual flows are aggregated in classes.

Benefits of DiffServ

The DiffServ architecture is an elegant way to provide much needed service discrimination within a commercial network. Customers willing to pay more will see

their applications receive better service than those paying less. This scheme exhibits an "auto-funding" property: "popular" traffic classes generate more revenues, which can be used to increase their provisioning.

A traffic class is a predefined aggregate of traffic. Compared with the aggregate of flows described earlier, traffic classes in DiffServ are accessible without signaling, which means they are readily available to applications without any setup delay. Consequently, traffic classes can provide *qualitative* or *relative* services to applications that cannot express their requirements quantitatively. This conforms to the original design philosophy of the Internet. An example of qualitative service is "traffic offered at service level A will be delivered with low latency," while a relative service could be "traffic offered at service level A will be delivered with higher probability than traffic offered at service level B." *Quantitative* services can also be provided by DiffServ. A quantitative service might be "90 percent of in-profile traffic offered at service level C will be delivered."

Since the provisioning of traffic classes is left to the provider's discretion, this provisioning can, and in the near future will, be performed statically and manually. Hence, existing management tools and protocols can be used to that end. However, this does not rule out the possibility of more automatic procedures for provisioning.

The only functionality actually imposed by DiffServ in interior routers is packet classification. This classification is simplified from that in RSVP because it is based on a single IP header field containing the DS codepoint, rather than multiple fields from different headers. This has the potential of allowing functions performed on every packet, such as traffic policing or shaping, to be done at the boundaries of domains, so forwarding is the main operation performed within the provider network.

Another advantage of DiffServ is that the classification of the traffic, and the subsequent selection of a DS codepoint for the packets, need not be performed in the end systems. Indeed, any router in the stub network where the host resides, or the ingress router at the boundary between the stub and provider networks, can be configured to classify (on a per-flow basis), mark, and shape the traffic from the hosts. Such routers are the only points where per-flow classification may occur, which does not pose any problem because they are at the edge of the Internet, where flow concentration is low. The

potential noninvolvement of end systems, and the use of existing and widespread management tools and protocols allows swift and incremental deployment of the DiffServ architecture.

Short comings of DiffServ

Simultaneously providing several services with differing qualities within the same network is a very difficult task. Despite its apparent simplicity, DiffServ does not make this task any simpler. *Instead, in DiffServ it was decided to keep the operating mode of the network simple by pushing as much complexity as possible onto network provisioning and configuration.* Network provisioning and configuration require sophisticated tools and traffic models. So far, large networks have mainly offered a single type of service (best-effort service in the Internet, interactive voice in telephone networks, etc.). The provisioning of networks providing multiple classes of service at the same time is therefore a rather new area and quite challenging due to possibly adverse interactions between different classes of service. The construction of end-to-end services by concatenating local service agreements is also a nontrivial issue.

The key to provisioning is the knowledge of traffic patterns and volumes traversing *each* node of the network. This also requires a good knowledge of network topology and routing. The problem with the Internet is that provisioning will be performed on a much slower time scale than the time scales at which traffic dynamics and network dynamics (e.g., route changes) occur. This problem can be illustrated with the simplest case of a single service provider network whose service agreements with customers are static. Although the amount of traffic entering the domain is known and policed, it is impossible to guarantee that overloading of resources will be avoided. This is caused by two factors:

- The entering packets can be bound to any destination in the Internet, and may thus be routed towards any border router of the domain (except the one where it entered). In the worst case, a substantial proportion of the entering packets might all exit the domain through the same border router
- Route changes can suddenly shift vast amounts of traffic from one router to another.

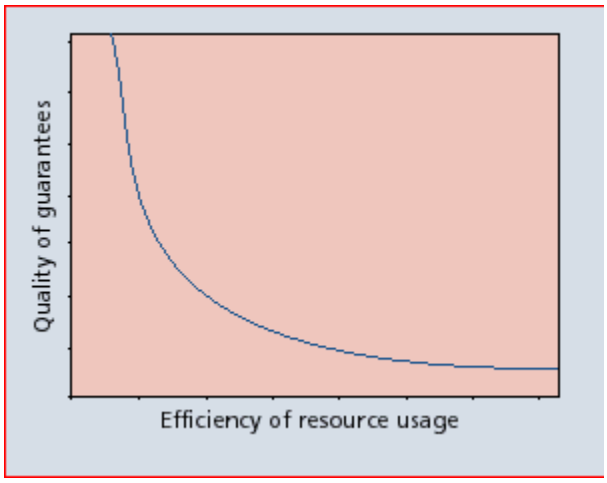


Figure 7 - The trade-off between quality and efficiency.

Therefore, unless resources are massively over provisioned in both interior and border routers, traffic and network dynamics can cause momentary violation of service agreements, especially those relating to quantitative services. On the other hand, massive over provisioning results in a very poor statistical multiplexing gain, and is therefore inefficient and expensive, as illustrated in Figure 7.

To increase resource usage in the network, service providers can trade generality and robustness for efficiency. For example, to limit the amount of expensive resources dedicated to the support of quantitative services, service providers can limit quantitative service contracts to apply between any pair of border routers in the domain. In such a case, the service would apply only to packets entering the domain at a designated ingress router and leaving the domain at a designated egress router. This helps solve the first problem described above at the cost of generality, since only packets bound for destinations "served" through the egress router can benefit from the service. To ensure that the egress router is in the route to any given destination, the interdomain routing entry for that destination must be statically fixed in the ingress router. Even for a fixed ingress-egress pair, intradomain routing dynamics can still occur. This means that the set of internal routers visited by the packets travelling between the ingress and egress routers can still suddenly change. However, the "directionality" of the traffic considered here is such that the number of possible routes is considerably reduced compared with the general case, and so is the resulting and necessary over provisioning. A service provider could, however, reduce to a minimum the over provisioning of quantitative services

offered between pairs of border routers by "pinning" the intradomain route between those routers. Fixing the egress router for a given destination and/or pinning internal routes between border routers nevertheless incurs a loss of robustness. In multicast, where receivers can join and leave the communication at any time, the problem of efficient provisioning will be even worse.

Alternatively, a service provider might wish to use dynamic logical provisioning and configuration (i.e., sharing of resources between classes) as an answer to the problems of network and traffic dynamics. However, depending on the type of service agreement (qualitative, relative, or quantitative) and the QoS parameters involved in the agreement, dynamic logical provisioning might require signaling and admission control.

From the point of view of a flow, the class bandwidth is not a meaningful parameter. In fact, bandwidth is a class property *shared* by all the flows in the class, and the bandwidth received by an individual flow depends on the number of competing flows in the class as well as the fairness of their respective responses to traffic conditions in the class. Therefore, to receive some quantitative bandwidth guarantees, a flow must "reserve" its share of bandwidth along the data path, which involves some form of end-to-end signaling and admission control (at least among logical entities called *bandwidth brokers*). This end-to-end signaling should also track network dynamics (i.e., route changes) to enforce the guarantees, which can be very complex. Furthermore, even qualitative bandwidth agreements require end-to-end signaling and admission control. This is because even if one class is guaranteed to have more bandwidth than another, the number and behavior of flows in the latter class may result in smaller shares of bandwidth for these flows than for the flows in the other class. Hence, in this case, end-to-end signaling would also be required to ensure that in every node along the path, the bandwidth received by a flow in a high bandwidth class is greater than the bandwidth received by a flow in a smaller bandwidth class.

On the other hand, delay and error rates are class properties that *apply* to every flow of a class. This is because in every router visited, all the packets sent in a given class share the queue devoted to that class. Consequently, as long as each router manages its queues to maintain a relative relationship between the delay and/or error rate of different classes, relative service agreements can be guaranteed without any signaling. However, if

quantitative delay or error rate bounds are required, end-to-end signaling and admission control are also required.

End-to-end signaling and admission control would increase the complexity of the DiffServ architecture. The idea of dynamically negotiable service agreements has also been suggested as a way of improving resource usage in the network. Such dynamic service-level agreements would require complex signaling, since the changes might affect the agreements a provider has with several neighboring networks. The time scale on which such dynamic provisioning could occur would be limited by scalability considerations, which in turn could impede its usefulness.

It is therefore believed that in its simplest and most general form, DiffServ can efficiently provide pure relative service agreements on delay and error rates among classes. However, unless complex signaling and admission control are introduced in the DiffServ architecture, or generality and robustness are sacrificed to some extent, guarantees on bandwidth as well as quantitative bounds on delay and error rates cannot be provided. It is should kept in mind that signaling could only be used by applications which can at least give a quantitative estimate of their requirements, and reside on hosts that have been modified to support signaling, thus limiting the immediate usability of DiffServ.

It should be noted that from a complexity point of view, a DiffServ scenario with dynamic provisioning and admission control is very close to an IntServ scenario with flow aggregation. The difference is that precise delay and error rate bounds might not be computed with DiffServ, since the delays and error rates introduced by each router in the domain may not be available to the bandwidth broker.

Consequently, although DiffServ will undoubtedly improve support for a number of applications and is urgently needed in the Internet, it does not represent the ultimate solution for QoS support for all types of applications. The suitability of DiffServ for a given application could also depend on the context in which that application is being used. If we take the example of Internet telephony, we see that DiffServ is suitable for providing a cheap solution for internal calls between remote sites of a company by emulating leased lines between these sites. However, DiffServ may prove unsuitable for the support of telephony over the Internet for the general public, because people do not usually restrict their calls to only a few destinations.

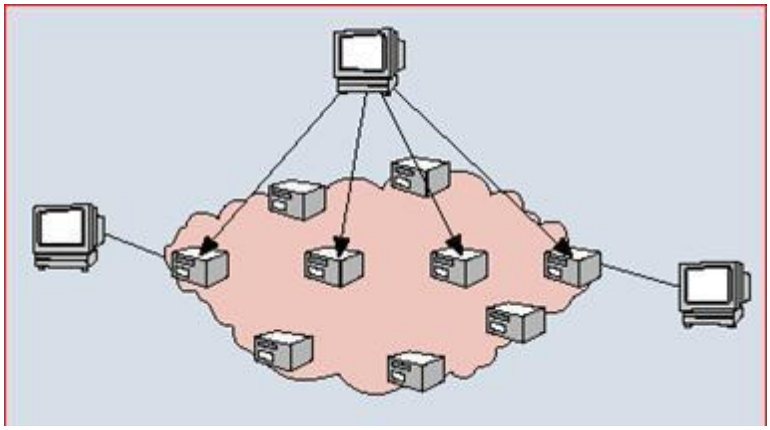


Figure 8 - Top-down provisioning of a differentiated services network

The current direction of the diffserv working group calls for the provisioning and configuration of the diffserv network to be done in a *top-down* manner, as illustrated in Figure 8. A final note is that Diff-Serv lays a valuable foundation for IP QoS, but it cannot provide an end-to-end QoS architecture by itself. Diff-Serv is a class-based framework, and an additional set of requirements must be addressed to achieve a real implementation with hard guarantees, including:

- A set of DS field code points in lieu of standards.
- Quantitative descriptions of class performance attributes.
- A mechanism for efficiently aggregating many sources of premium class traffic that can converge at transit routers.
- A solution to the single-ended service level agreement (SLA) problem.
- An interworking solution for mapping IP CoS to ATM QoS.
- Management tools to facilitate deployment and operation.

2.4. Multiprotocol Label Switching

Multiprotocol Label Switching (MPLS) is a routing technique that augments speed and control of the network traffic by directing data from one node to the next node based on short path labels. Instead of being routed using long network addresses, the data packets are routed through path labels that identify virtual paths between the nodes rather than endpoints. MPLS speeds up traffic flows by avoiding complex lookups in the routing table at each node as in conventional routing algorithms.

MPLS is a scalable and protocol-independent routing technique. It works with Internet Protocol (IP), Ethernet, Frame Relay and Asynchronous Transport Mode (ATM). Despite the advent of newer technologies, it remains relevant due to its features like security, flexibility and traffic engineering.

Working Principle

MPLS works by prefixing 32-bit labels with the MPLS header. The 32-bit label contains four fields −

- Label value field of 20-bits
- Traffic class field of 3-bits for QoS (quality of service)
- Bottom of stack flag of 1-bit (1 value denotes that the current label is the last one in the stack)
- TTL (time to live) field of 8-bits

When an IP packet enters the MPLS network, the 32-bit MPLS label is added by the ingress router, which is a label edge router (LER). LER decides the virtual path called label-switched path (LSP) that the packet will follow until it reaches its destination.

The subsequent label-switching routers (LSRs) along the LSP, forwards the packet based upon only the MPLS labels. They do not look beyond the MPLS label to the IP header.

Below is an overview of MPLS:

- It forwards using labels, as opposed to network addresses.
- The label contains the service class, as well as the destination, of the packet.
- It operates between Layers 2 and 3 of the Open Systems Interconnection (OSI) model.
- It guarantees the bandwidth of paths.
- ATM switches can act as routers, so no additional hardware is needed.

How an MPLS network works

In an MPLS network, packets are labeled by an ingress router -- a label edge router (LER) -- as they enter a service provider's network. The first router to receive a packet calculates the packet's entire path upfront. It also conveys a unique identifier to subsequent routers using a label in the packet header.

Every prefix in a routing table receives a unique identifier, and the MPLS service tells routers exactly where to look in the routing table for a specific prefix. This mechanism speeds up communication and traffic hopping.

MPLS works between the following OSI model layers:

- **Layer 2.** The data-link layer, or switching level, which uses protocols such as Ethernet.
- **Layer 3.** The routing layer, which covers traffic routing.

MPLS label traffic is sent via a label-switched path (LSP) inserted between the Layer 2 and Layer 3 headers. Label switch routers (LSRs) interpret the MPLS labels -- not the full IP address of any traffic. MPLS forwards data packets to Layer 2 of the OSI model, rather than passing to Layer 3. For this reason, MPLS is informally described as operating at Layer 2.5.

MPLS routing terminology

Label edge routers. LERs are the ingress or egress routers or nodes when an LSR is the first or last router in the path, respectively. LSRs label incoming data -- the ingress node -- or pop the label off the packet.

Label-switched paths. LSPs are the pathways through which packets are routed. An LSP enables service providers to decide the best way to flow certain types of traffic within a private or public network.

Label switch routers. LSRs read the labels and send labeled data on identified pathways. Intermediate LSRs are available if a packet data link needs to be corrected.

Pop. This mechanism removes a label and is usually performed by the egress router.

Push. This mechanism adds a label and is typically performed by the ingress router.

Swap. This mechanism replaces a label and is usually performed by LSRs between the ingress and egress routers.

Steps of an MPLS network traffic pathway

Here is an example of how a packet travels through an MPLS network:

1. A packet enters the network through an LER.
2. The packet is assigned to a forwarding equivalence class (FEC). The FEC assignment depends on the type of data and the destination. FECs are used to identify packets with similar or identical characteristics.
3. The LER -- or ingress node -- applies a label to the packet and pushes it inside an LSP. The LER decides on which LSP the packet takes until it reaches its destination address.
4. The packet moves through the network across LSRs.
5. When an LSR receives a packet, it carries out the Push, Swap and Pop actions.
6. In the final step, the LSR -- or egress router -- removes the labels and then forwards the original IP packet toward its destination.

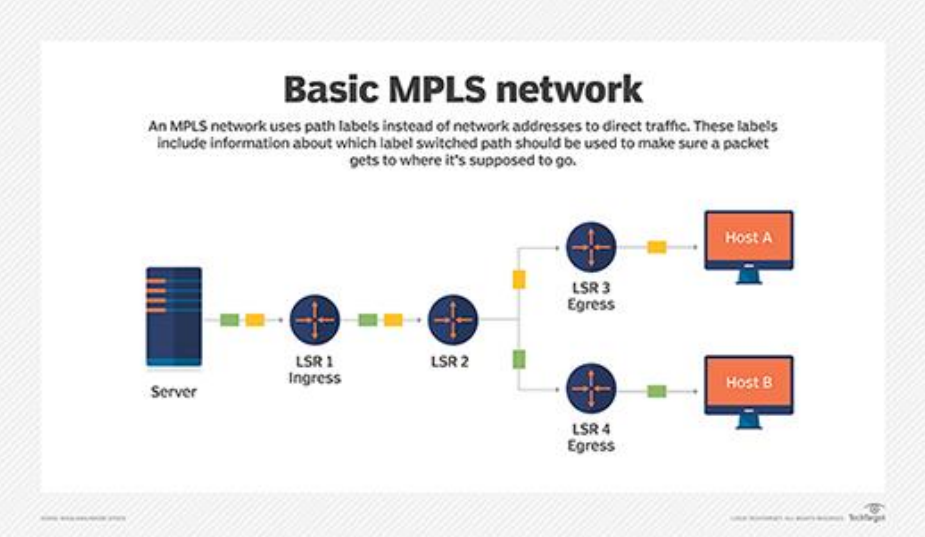


Figure.9 MPLS Network

Benefits of MPLS

Router hardware has improved significantly since the development of MPLS, but MPLS still offers important benefits.

QoS controls and reliability

Services need to be able to meet service-level agreements that cover traffic latency, jitter, packet loss and downtime. Service providers and enterprises use MPLS to implement QoS by defining LSPs that can meet the specific needs of a service. For example, a network might offer three service levels, each prioritizing different types of traffic -- e.g., one level for voice, one for time-sensitive traffic and one for best-effort traffic.

VPNs

MPLS supports traffic separation and the creation of virtual private networks (VPNs), virtual private local area network services and virtual leased lines.

Agnostic protocol support

MPLS is not tied to any specific protocol or transport medium. MPLS supports transport over IP, Ethernet, ATM and frame relay. An LSP can be created using any protocol. Generalized MPLS extends MPLS, which manages time-division multiplexing, and other classes of switching technologies beyond packet switching.

Reduced latency and improved performance

MPLS is ideal for latency-sensitive applications, such as those handling videos, voice and mission-critical data. In addition, MPLS reduces latency by routing data more quickly using shorter path labels.

To optimize performance, different types of data can be preprogrammed with other priorities and classes of service. Organizations can assign different bandwidth percentages for various kinds of data to ensure optimal delivery and access.

Scalability

MPLS networks are scalable. Companies can provision and pay for only the bandwidth they need until their requirements change.

2.5. Virtual Private Networks (VPN) and Tunnels

What is tunneling?

In the physical world, tunneling is a way to cross terrain or boundaries that could not normally be crossed. Similarly, in networking, tunnels are a method for transporting data across a network using protocols that are not supported by that network. Tunneling works by encapsulating packets: wrapping packets inside of other packets. (Packets are small pieces of data that can be re-assembled at their destination into a larger file.)

Tunneling is often used in virtual private networks (VPNs). It can also set up efficient and secure connections between networks, enable the usage of unsupported network protocols, and in some cases allow users to bypass firewalls.

One way to build virtual private networks (VPNs) is to use tunnels. It turns out that MPLS can be thought of as a way to build tunnels, and this makes it suitable for building VPNs of various types.

The simplest form of MPLS VPN to understand is a layer 2 VPN. In this type of VPN, MPLS is used to tunnel layer 2 data (such as Ethernet frames or ATM cells) across a network of MPLS-enabled routers. One reason for tunnels is to provide some sort of network service (such as multicast) that is not supported by some routers in the network. The same logic applies here: IP routers are not ATM switches, so you cannot provide an ATM virtual circuit service across a network of conventional routers. However, if you had a pair of routers interconnected by a tunnel, they could send ATM cells across the tunnel and emulate an ATM circuit. The term for this technique within the IETF is *pseudowire emulation*. Figure 15 illustrates the idea.

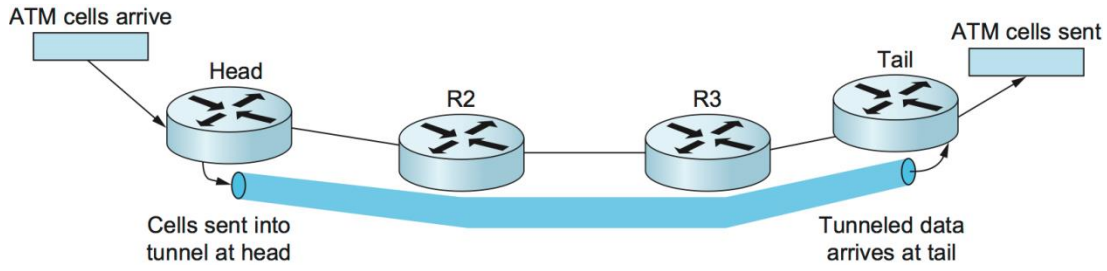


Figure 15. An ATM circuit is emulated by a tunnel.

We have already seen how IP tunnels are built: The router at the entrance of the tunnel wraps the data to be tunneled in an IP header (the *tunnel header*), which represents the address of the router at the far end of the tunnel and sends the data like any other IP packet. The receiving router receives the packet with its own address in the header, strips the tunnel header, and finds the data that was tunneled, which it then processes. Exactly what it does with that data depends on what it is. For example, if it were another IP packet, it would then be forwarded on like a normal IP packet. However, it need not be an IP packet, as long as the receiving router knows what to do with non-IP packets. We'll return to the issue of how to handle non-IP data in a moment.

An MPLS tunnel is not too different from an IP tunnel, except that the tunnel header consists of an MPLS header rather than an IP header. Looking back to our first example, in Figure 15, we saw that router R1 attached a label (15) to every packet that it sent towards prefix 18.1.1. Such a packet would then follow the path R1-R2-R3, with each router in the path examining only the MPLS label. Thus, we observe that there was no requirement that R1 only send IP packets along this path—any data could be wrapped up in the MPLS header and it would follow the same path, because the intervening routers never look beyond the MPLS header. In this regard, an MPLS header is just like an IP tunnel header (except only 4 bytes long instead of 20 bytes). The only issue with sending non-IP traffic along a tunnel, MPLS or otherwise, is what to do with non-IP traffic when it reaches the end of the tunnel. The general solution is to carry some sort of demultiplexing identifier in the tunnel payload that tells the router at the end of the tunnel what to do. It turns out that an MPLS label is a perfect fit for such an identifier. An example will make this clear.

Let's assume we want to tunnel ATM cells from one router to another across a network of MPLS-enabled routers, as in Figure 11. Further, we assume that the goal is to emulate an ATM virtual circuit; that is, cells arrive at the entrance, or head, of the tunnel on a certain input port with a certain VCI and should leave the tail end of the tunnel on a certain output port and potentially different VCI. This can be accomplished by configuring the head and tail routers as follows:

- The head router needs to be configured with the incoming port, the incoming VCI, the demultiplexing label for this emulated circuit, and the address of the tunnel end router.

- The tail router needs to be configured with the outgoing port, the outgoing VCI, and the demultiplexing label.

Once the routers are provided with this information, we can see how an ATM cell would be forwarded. Figure 15 illustrates the steps.

1. An ATM cell arrives on the designated input port with the appropriate VCI value (101 in this example).
2. The head router attaches the demultiplexing label that identifies the emulated circuit.
3. The head router then attaches a second label, which is the tunnel label that will get the packet to the tail router. This label is learned by mechanisms just like those described elsewhere in this section.
4. Routers between the head and tail forward the packet using only the tunnel label.
5. The tail router removes the tunnel label, finds the demultiplexing label, and recognizes the emulated circuit.
6. The tail router modifies the ATM VCI to the correct value (202 in this case) and sends it out the correct port.

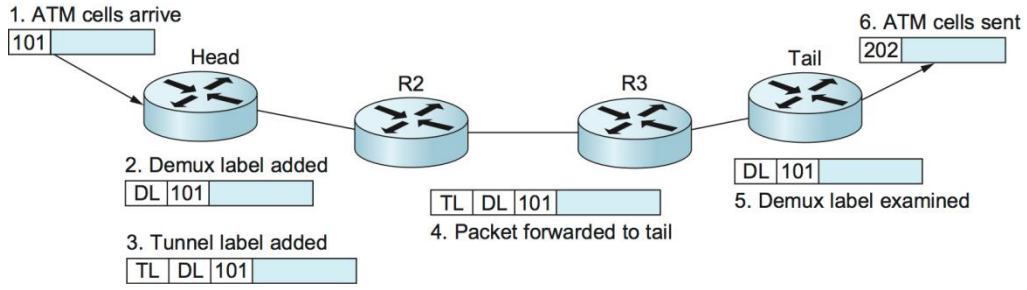


Figure 16. Forward ATM cells along a tunnel.

One item in this example that might be surprising is that the packet has two labels attached to it. This is one of the interesting features of MPLS—labels may be stacked on a packet to any depth. This provides some useful scaling capabilities. In this example, it allows a single tunnel to carry a potentially large number of emulated circuits.

The same techniques described here can be applied to emulate many other layer 2 services, including Frame Relay and Ethernet. It is worth noting that virtually identical capabilities can be provided using IP tunnels; the main advantage of MPLS here is the shorter tunnel header.

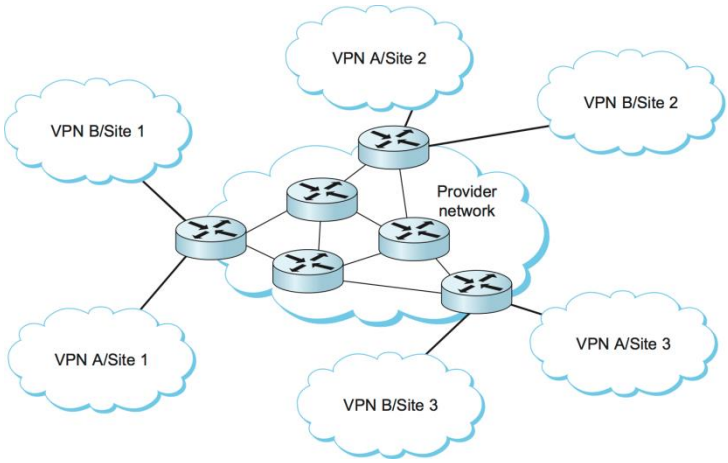


Figure 17. Example of a layer 3 VPN. Customers A and B each obtain a virtually private IP service from a single provider.

Before MPLS was used to tunnel layer 2 services, it was also being used to support layer 3 VPNs. We won't go into the details of layer 3 VPNs, which are quite complex, but we will note that they represent one of the most popular uses of MPLS today. Layer 3 VPNs also use stacks of MPLS labels to tunnel packets across an IP network. However, the packets that are tunneled are themselves IP packets—hence, the name *layer 3* VPNs. In a layer 3 VPN, a single service provider operates a network of MPLS-enabled routers and provides a "virtually private" IP network service to any number of distinct customers. That is, each customer of the provider has some number of sites, and the service provider creates the illusion for each customer that there are no other customers on the network. The customer sees an IP network interconnecting his own sites and no other sites. This means that each customer is isolated from all other customers in terms of both routing and addressing. Customer A can't sent packets directly to customer B, and *vice versa*. Customer A can even use IP addresses that have also been used by customer B. The basic idea is illustrated in Figure 17. As in layer 2 VPNs, MPLS is used to tunnel packets from one site to another; however, the configuration of the tunnels is performed automatically by some fairly elaborate use of BGP, which is beyond the scope of this book.

Customer A in fact usually *can* send data to customer B in some restricted way. Most likely, both customer A and customer B have some connection to the global Internet, and thus it is probably possible for customer A to send email messages, for example, to the mail server inside customer B's network. The "privacy" offered by a VPN prevents customer A from having unrestricted access to all the machines and subnets inside customer B's network.

In summary, MPLS is a rather versatile tool that has been applied to a wide range of different networking problems. It combines the label-swapping forwarding mechanism that is normally associated with virtual circuit networks with the routing and control protocols of IP datagram networks to produce a class of network that is somewhere between the two conventional extremes. This extends the capabilities of IP networks to enable, among other things, more precise control of routing and the support of a range of VPN services.

Types of Virtual Private Network (VPN)

VPN stands for Virtual Private Network (VPN), that allows a user to connect to a private network over the Internet securely and privately. VPN creates an encrypted connection that is called VPN tunnel, and all Internet traffic and communication is passed through this secure tunnel. Virtual Private Network (VPN) is basically of 2 types:

1. Remote Access VPN

Remote Access VPN permits a user to connect to a private network and access all its services and resources remotely. The connection between the user and the private network occurs through the Internet and the connection is secure and private. Remote Access VPN is useful for home users and business users both. An employee of a company, while he/she is out of station, uses a VPN to connect to his/her company's private network and remotely access files and resources on the private network. Private users or home users of VPN, primarily use VPN services to bypass regional restrictions on the Internet and access blocked websites. Users aware of Internet security also use VPN services to enhance their Internet security and privacy.

2. Site to Site VPN

A Site-to-Site VPN is also called as Router-to-Router VPN and is commonly used in the large companies. Companies or organizations, with branch offices in different locations, use Site-to-site VPN to connect the network of one office location to the network at another office location.

- **Intranet based VPN:** When several offices of the same company are connected using Site-to-Site VPN type, it is called as Intranet based VPN.

- **Extranet based VPN:** When companies use Site-to-site VPN type to connect to the office of another company, it is called as Extranet based VPN.

3. Cloud VPN

A Cloud VPN is a virtual private network that allows users to securely connect to a cloud-based infrastructure or service. It uses the internet as the primary transport medium to connect the remote users to the cloud-based resources. Cloud VPNs are typically offered as a service by cloud providers such as Amazon Web Services (AWS) and Microsoft Azure. It uses the same encryption and security protocols as traditional VPNs, such as IPsec or SSL, to ensure that the data transmitted over the VPN is secure. Cloud VPNs are often used by organizations to securely connect their on-premises resources to cloud-based resources, such as cloud-based storage or software-as-a-service (SaaS) applications.

4. Mobile VPN

Mobile VPN is a virtual private network that allows mobile users to securely connect to a private network, typically through a cellular network. It creates a secure and encrypted connection between the mobile device and the VPN server, protecting the data transmitted over the connection. Mobile VPNs can be used to access corporate resources, such as email or internal websites, while the user is away from the office. They can also be used to securely access public Wi-Fi networks, protecting the user's personal information from being intercepted. Mobile VPNs are available as standalone apps or can be integrated into mobile device management (MDM) solutions. These solutions are commonly used by organisations to secure their mobile workforce.

5. SSL VPN

SSL VPN (Secure Sockets Layer Virtual Private Network) is a type of VPN that uses the SSL protocol to secure the connection between the user and the VPN server. It allows remote users to securely access a private network by establishing an encrypted tunnel between the user's device and the VPN server. SSL VPNs are typically accessed through a web browser, rather than through a standalone client. This makes them easier to use and deploy, as they don't require additional software to be installed on the user's device. It can be used to access internal resources such as email, file servers, or databases. SSL VPNs are considered more secure than traditional IPsec VPNs because they use the same encryption protocols as HTTPS, the secure version of HTTP used for online transactions.

6. PPTP (Point-to-Point Tunneling Protocol) VPN

PPTP (Point-to-Point Tunneling Protocol) is a type of VPN that uses a simple and fast method for implementing VPNs. It creates a secure connection between two computers by encapsulating the data packets being sent between them. PPTP is relatively easy to set up and doesn't require any additional software to be installed on the client's device. It can be used to access internal

resources such as email, file servers, or databases. PPTP is one of the oldest VPN protocols and is supported on a wide range of operating systems. However, it is considered less secure than other VPN protocols such as L2TP or OpenVPN, as it uses a weaker encryption algorithm and has been known to have security vulnerabilities.

7. L2TP (Layer 2 Tunneling Protocol) VPN

L2TP (Layer 2 Tunneling Protocol) is a type of VPN that creates a secure connection by encapsulating data packets being sent between two computers. L2TP is an extension of PPTP, it adds more security to the VPN connection by using a combination of PPTP and L2F (Layer 2 Forwarding Protocol) and it uses stronger encryption algorithm than PPTP. L2TP is relatively easy to set up and doesn't require additional software to be installed on the client's device. It can be used to access internal resources such as email, file servers, or databases. It is supported on a wide range of operating systems, but it is considered less secure than other VPN protocols such as OpenVPN, as it still has some vulnerabilities that can be exploited.

8. OpenVPN

OpenVPN is an open-source software application that uses SSL and is highly configurable and secure. It creates a secure and encrypted connection between two computers by encapsulating the data packets being sent between them. OpenVPN can be used to access internal resources such as email, file servers, or databases. It is supported on a wide range of operating systems and devices, and can be easily configured to work with various network configurations and security settings. It is considered one of the most secure VPN protocols as it uses the industry standard SSL/TLS encryption protocols and it offers advanced features such as two-factor authentication and kill switch.

How does packet encapsulation work?

Data traveling over a network is divided into packets. A typical packet has two parts: the header, which indicates the packet's destination and which protocol it uses, and the payload, which is the packet's actual contents.

An encapsulated packet is essentially a packet inside another packet. In an encapsulated packet, the header and payload of the first packet goes inside the payload section of the surrounding packet. The original packet itself becomes the payload.

Why is encapsulation useful?

All packets use networking protocols — standardized ways of formatting data — to get to their destinations. However, not all networks support all protocols. Imagine a company wants to set up a wide area network (WAN) connecting Office A and Office B. The company uses the IPv6 protocol, which is the latest version of the Internet Protocol (IP), but there is a network between

Office A and Office B that only supports IPv4. By encapsulating their IPv6 packets inside IPv4 packets, the company can continue to use IPv6 while still sending data directly between the offices.

Encapsulation is also useful for encrypted network connections. *Encryption* is the process of scrambling data in such a way that it can only be unscrambled using a secret encryption key; the process of undoing encryption is called *decryption*. If a packet is completely encrypted, including the header, then network routers will not be able to forward the packet to its destination since they do not have the key and cannot see its header. By wrapping the encrypted packet inside another unencrypted packet, the packet can travel across networks like normal.

What is a VPN tunnel?

A VPN is a secure, encrypted connection over a publicly shared network. Tunneling is the process by which VPN packets reach their intended destination, which is typically a private network.

Many VPNs use the IPsec protocol suite. IPsec is a group of protocols that run directly on top of IP at the network layer. Network traffic in an IPsec tunnel is fully encrypted, but it is decrypted once it reaches either the network or the user device. (IPsec also has a mode called "transport mode" that does not create a tunnel.)

Another protocol in common use for VPNs is Transport Layer Security (TLS). This protocol operates at either layer 6 or layer 7 of the OSI model depending on how the model is interpreted. TLS is sometimes called SSL (Secure Sockets Layer), although SSL refers to an older protocol that is no longer in use.

What is split tunneling?

Usually, when a user connects their device to a VPN, all their network traffic goes through the VPN tunnel. Split tunneling allows some traffic to go outside of the VPN tunnel. In essence, split tunneling lets user devices connect to two networks simultaneously: one public and one private.

What is GRE tunneling?

Generic Routing Encapsulation (GRE) is one of several tunneling protocols. GRE encapsulates data packets that use one routing protocol inside the packets of another protocol. GRE is one way to set up a direct point-to-point connection across a network, for the purpose of simplifying connections between separate networks.

GRE adds two headers to each packet: the GRE header and an IP header. The GRE header indicates the protocol type used by the encapsulated packet. The IP header encapsulates the original packet's IP header and payload. Only the routers at each end of the GRE tunnel will reference the original, non-GRE IP header.

What is IP-in-IP?

IP-in-IP is a tunneling protocol for encapsulating IP packets inside other IP packets. IP-in-IP does not encrypt packets and is not used for VPNs. Its main use is setting up network routes that would not normally be available.

What is SSH tunneling?

The Secure Shell (SSH) protocol sets up encrypted connections between client and server, and can also be used to set up a secure tunnel. SSH operates at layer 7 of the OSI model, the application layer. By contrast, IPsec, IP-in-IP, and GRE operate at the network layer.

What are some other tunneling protocols?

In addition to GRE, IPsec, IP-in-IP, and SSH, other tunneling protocols include:

Types of Virtual Private Network (VPN) Protocols:

1. **Internet Protocol Security (IPSec):** Internet Protocol Security, known as IPSec, is used to secure Internet communication across an IP network. IPSec secures Internet Protocol communication by verifying the session and encrypts each data packet during the connection. IPSec runs in 2 modes:
    - (i) Transport mode
    - (ii) Tunneling mode
2. **Layer 2 Tunneling Protocol (L2TP):** L2TP or Layer 2 Tunneling Protocol is a tunneling protocol that is often combined with another VPN security protocol like IPSec to establish a highly secure VPN connection. L2TP generates a tunnel between two L2TP connection points and IPSec protocol encrypts the data and maintains secure communication between the tunnel.
3. **Point–to–Point Tunneling Protocol (PPTP):** PPTP or Point-to-Point Tunneling Protocol generates a tunnel and confines the data packet. Point-to-Point Protocol (PPP) is used to encrypt the data between the connection. PPTP is one of the most widely used VPN protocol and has been in use since the early release of Windows. PPTP is also used on Mac and Linux apart from Windows.
4. **SSL and TLS:** SSL (Secure Sockets Layer) and TLS (Transport Layer Security) generate a VPN connection where the web browser acts as the client and user access is prohibited to specific applications instead of entire network. Online shopping websites commonly uses SSL and TLS protocol. It is easy to switch to SSL by web browsers and with almost

no action required from the user as web browsers come integrated with SSL and TLS. SSL connections have “https” in the initial of the URL instead of “http”.

5. **Secure Shell (SSH):** Secure Shell or SSH generates the VPN tunnel through which the data transfer occurs and also ensures that the tunnel is encrypted. SSH connections are generated by a SSH client and data is transferred from a local port on to the remote server through the encrypted tunnel.
6. **SSTP (Secure Socket Tunneling Protocol):** A VPN protocol developed by Microsoft that uses SSL to secure the connection, but only available for Windows.
7. **IKEv2 (Internet Key Exchange version 2):** A VPN protocol that provides fast and secure connections, but not widely supported by VPN providers.
8. **OpenVPN:** An open-source VPN protocol that is highly configurable and secure, widely supported by VPN providers and considered one of the most secure VPN protocols.
9. **WireGuard:** A relatively new and lightweight VPN protocol that aims to be faster, simpler and more secure than existing VPN protocols.

How does Cloudflare use tunneling?

Cloudflare Magic Transit protects on-premise, cloud, and hybrid network infrastructure from DDoS attacks and other threats. In order for Magic Transit to work, the Cloudflare network has to be securely connected to the customer's internal network. Cloudflare uses GRE tunneling to form these connections. With GRE tunneling, Magic Transit is able to connect directly to Cloudflare customers' networks securely over the public Internet.

2.6. Destination-Based Forwarding:

A very similar idea is now implemented in MPLS-enabled routers. The following example shows how this idea works.

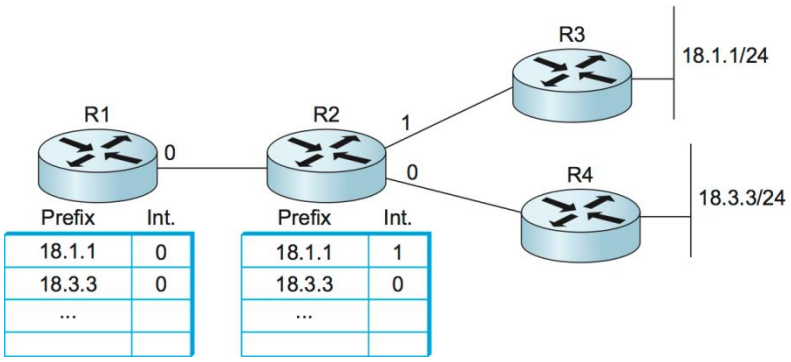


Figure 10. Routing tables in example network.

Consider the network in Figure 10. Each of the two routers on the far right (R3 and R4) has one connected network, with prefixes 18.1.1/24 and 18.3.3/24. The remaining routers (R1 and R2) have routing tables that indicate which outgoing interface each router would use when forwarding packets to one of those two networks.

When MPLS is enabled on a router, the router allocates a label for each prefix in its routing table and advertises both the label and the prefix that it represents to its neighboring routers. This advertisement is carried in the Label Distribution Protocol. This is illustrated in Figure 11. Router R2 has allocated the label value 15 for the prefix 18.1.1 and the label value 16 for the prefix 18.3.3. These labels can be chosen at the convenience of the allocating router and can be thought of as indices into the routing table. After allocating the labels, R2 advertises the label bindings to its neighbors; in this case, we see R2 advertising a binding between the label 15 and the prefix 18.1.1 to R1. The meaning of such an advertisement is that R2 has said, in effect, "Please attach the label 15 to all packets sent to me that are destined to prefix 18.1.1." R1 stores the label in a table alongside the prefix that it represents as the remote or outgoing label for any packets that it sends to that prefix.

In Figure 11(c), we see another label advertisement from router R3 to R2 for the prefix 18.1.1, and R2 places the remote label that it learned from R3 in the appropriate place in its table.

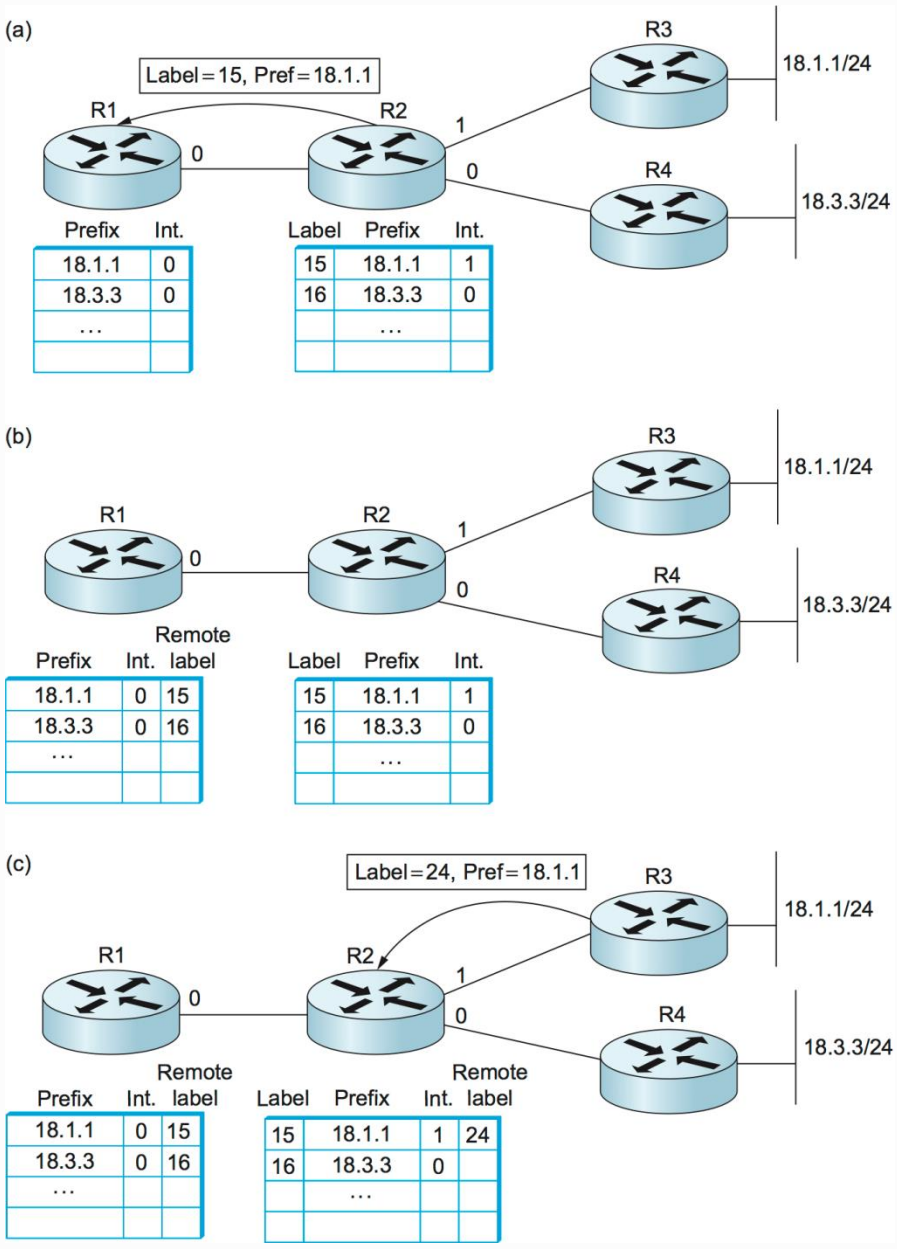


Figure 11. (a) R2 allocates labels and advertises bindings to R1. (b) R1 stores the received labels in a table. (c) R3 advertises another binding, and R2 stores the received label in a table.

At this point, we can look at what happens when a packet is forwarded in this network. Suppose a packet destined to the IP address 18.1.1.5 arrives from the left to router R1. R1 in this case is referred to as a *Label Edge Router* (LER); an LER performs a complete IP lookup on arriving IP packets and then applies labels to them as a result of the lookup. In this case, R1 would see that 18.1.1.5 matches the prefix 18.1.1 in its forwarding table and that this entry contains both an outgoing interface and a remote label value. R1 therefore attaches the remote label 15 to the packet before sending it.

When the packet arrives at R2, R2 looks only at the label in the packet, not the IP address. The forwarding table at R2 indicates that packets arriving with a label value of 15 should be sent out interface 1 and that they should carry the label value 24, as advertised by router R3. R2 therefore rewrites, or swaps, the label and forwards it on to R3.

What has been accomplished by all this application and swapping of labels? Observe that when R2 forwarded the packet in this example it never actually needed to examine the IP address. Instead, R2 looked only at the incoming label. Thus, we have replaced the normal IP destination address lookup with a label lookup. To understand why this is significant, it helps to recall that, although IP addresses are always the same length, IP prefixes are of variable length, and the IP destination address lookup algorithm needs to find the *longest match*—the longest prefix that matches the high order bits in the IP address of the packet being forwarded. By contrast, the label forwarding mechanism just described is an *exact match* algorithm. It is possible to implement a very simple exact match algorithm, for example, by using the label as an index into an array, where each element in the array is one line in the forwarding table.

Note that, while the forwarding algorithm has been changed from longest match to exact match, the routing algorithm can be any standard IP routing algorithm (e.g., OSPF). The path that a packet will follow in this environment is the exact same path that it would have followed if MPLS were not involved: the path chosen by the IP routing algorithms. All that has changed is the forwarding algorithm.

An important fundamental concept of MPLS is illustrated by this example. Every MPLS label is associated with a *forwarding equivalence class* (FEC)—a set of packets that are to receive the same forwarding treatment in a particular router. In this example, each prefix in the routing table is an FEC; that is, all packets that match the prefix 18.1.1—no matter what the low order bits of the IP address are—get forwarded along the same path. Thus, each router can allocate one label that maps to 18.1.1, and any packet that contains an IP address whose high order bits match that prefix can be forwarded using that label.

As we will see in the subsequent examples, FECs are a very powerful and flexible concept. FECs can be formed using almost any criteria; for example, all the packets corresponding to a particular customer could be considered to be in the same FEC.

Returning to the example at hand, we observe that changing the forwarding algorithm from normal IP forwarding to label swapping has an important consequence: Devices that previously didn't know how to forward IP packets can be used to forward IP traffic in an MPLS network. The most notable early application of this result was to ATM switches, which can support MPLS without any changes to their forwarding hardware. ATM switches support the label-swapping forwarding algorithm just described, and by providing these switches with IP routing protocols and a method to distribute label bindings they could be turned into *Label Switching Routers* (LSRs)—devices that run IP control protocols but use the label switching forwarding algorithm. More recently, the same idea has been applied to optical switches.

Before we consider the purported benefits of turning an ATM switch into an LSR, we should tie up some loose ends. We have said that labels are "attached" to packets, but where exactly are they attached? The answer depends on the type of link on which packets are carried. Two common methods for carrying labels on packets are shown in Figure 12. When IP packets are carried as complete frames, as they are on most link types including Ethernet and PPP, the label is inserted as a "shim" between the layer 2 header and the IP (or other layer 3) header, as shown in the lower part of the figure. However, if an ATM switch is to function as an MPLS LSR, then the label needs to be in a place where the switch can use it, and that means it needs to be in the ATM cell header, exactly where one would normally find the virtual circuit identifier (VCI) and virtual path identifier (VPI) fields.

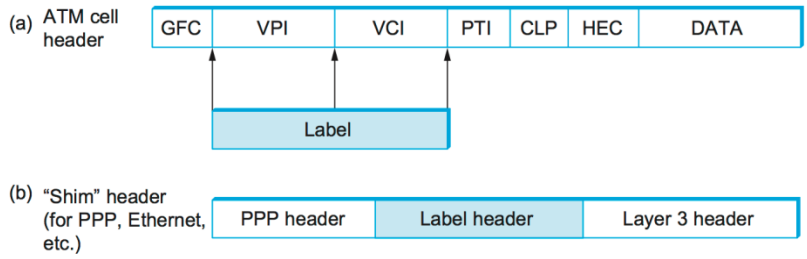


Figure 12. (a) Label on an ATM-encapsulated packet; (b) label on a frame-encapsulated packet.

Having now devised a scheme by which an ATM switch can function as an LSR, what have we gained? One thing to note is that we could now build a network that uses a mixture of conventional IP routers, label edge routers, and ATM switches functioning as LSRs, and they would all use the same routing protocols. To understand the benefits of using the same protocols, consider the alternative. In Figure 13(a), we see a set of routers interconnected by virtual circuits over an ATM network, a configuration called an *overlay* network. At one point in time, networks of this type were often built because commercially available ATM switches supported higher total throughput than routers. Today, networks like this are less common because routers have caught up with and even surpassed ATM switches. However, these networks still exist because of the significant installed base of ATM switches in network backbones, which in turn is partly a result of ATM's ability to support a range of capabilities such as circuit emulation and virtual circuit services.

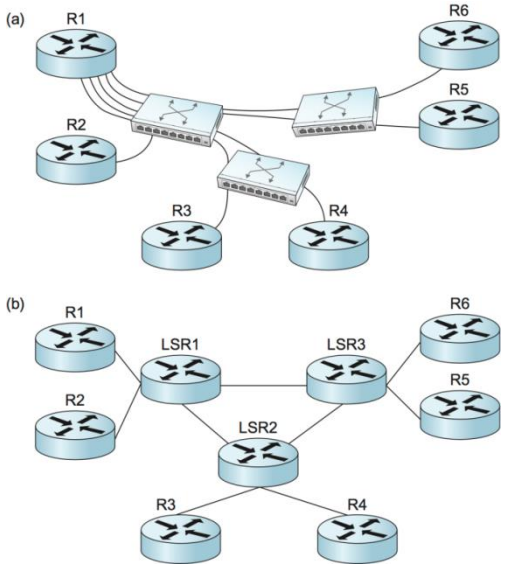


Figure 13. (a) Routers connect to each other using an overlay of virtual circuits. (b) Routers peer directly with LSRs.

In an overlay network, each router would potentially be connected to each of the other routers by a virtual circuit, but in this case for clarity we have just shown the circuits from R1 to all of its peer routers. R1 has five routing neighbors and needs to exchange routing protocol messages with all of them—we say that R1 has five routing adjacencies. By contrast, in Figure 13(b), the ATM switches have been replaced with LSRs. There are no longer virtual circuits interconnecting the routers. Thus, R1 has only one adjacency, with LSR1. In large networks, running MPLS on the switches leads to a significant reduction in the number of adjacencies that each router must maintain and can greatly reduce the amount of work that the routers have to do to keep each other informed of topology changes.

A second benefit of running the same routing protocols on edge routers and on the LSRs is that the edge routers now have a full view of the topology of the network. This means that if some link or node fails inside the network, the edge routers will have a better chance of picking a good new path than if the ATM switches rerouted the affected VCs without the knowledge of the edge routers.

Note that the step of "replacing" ATM switches with LSRs is actually achieved by changing the protocols running on the switches, but typically no change to the forwarding hardware is needed; that is, an ATM switch can often be converted to an MPLS LSR by upgrading only its software. Furthermore, an MPLS LSR might continue to support standard ATM capabilities at the same time as it runs the MPLS control protocols, in what is referred to as "ships in the night" mode.

The idea of running IP control protocols on devices that are unable to forward IP packets natively has been extended to Wavelength Division Multiplexing (WDM) and Time Division Multiplexing (TDM) networks (e.g., SONET). This is known as *Generalized MPLS* (GMPLS). Part of the motivation for GMPLS was to provide routers with topological knowledge of an optical network, just as in the ATM case. Even more important was the fact that there were no standard protocols for controlling optical devices, so MPLS proved to be a natural fit for that job.

2.6.1. Explicit Routing

IP has a source routing option, but it is not widely used for several reasons, including the fact that only a limited number of hops can be specified and because it is usual processed outside the "fast path" on most routers.

MPLS provides a convenient way to add capabilities similar to source-routing to IP networks, although the capability is more often referred to as *explicit routing* rather than *source routing*. One reason for the distinction is that it usually isn't the real source of the packet that picks the route. More often it is one of the routers inside a service provider's network. Figure 14 shows an example of how the explicit routing capability of MPLS might be applied. This sort of network is often called a *fish* network because of its shape (the routers R1 and R2 form the tail; R7 is at the head).

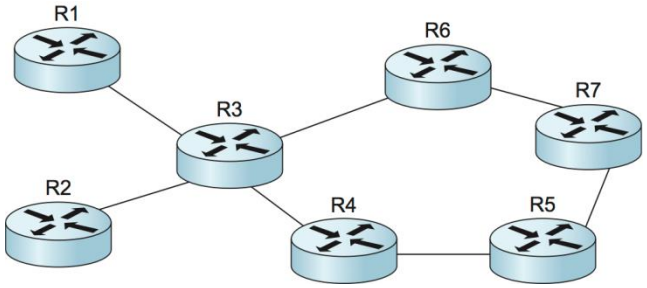


Figure 14. A network requiring explicit routing.

Suppose that the operator of the network in Figure 14 has determined that any traffic flowing from R1 to R7 should follow the path R1-R3-R6-R7 and that any traffic going from R2 to R7 should follow the path R2-R3-R4-R5-R7. One reason for such a choice would be to make good use of the capacity available along the two distinct paths from R3 to R7. We can think of the R1-to-R7 traffic as constituting one forwarding equivalence class, and the R2-to-R7 traffic constitutes a second FEC. Forwarding traffic in these two classes along different paths is difficult with normal IP routing, because R3 doesn't normally look at where traffic came from in making its forwarding decisions.

Because MPLS uses label swapping to forward packets, it is easy enough to achieve the desired routing if the routers are MPLS enabled. If R1 and R2 attach distinct labels to packets before sending them to R3—thus identifying them as being in different FECs—then R3 can forward

packets from R1 and R2 along different paths. The question that then arises is how do all the routers in the network agree on what labels to use and how to forward packets with particular labels? Clearly, we can't use the same procedures as described in the preceding section to distribute labels, because those procedures establish labels that cause packets to follow the normal paths picked by IP routing, which is exactly what we are trying to avoid. Instead, a new mechanism is needed. It turns out that the protocol used for this task is the Resource Reservation Protocol (RSVP). For now it suffices to say that it is possible to send an RSVP message along an explicitly specified path (e.g., R1-R3-R6-R7) and use it to set up label forwarding table entries all along that path. This is very similar to the process of establishing a virtual circuit.

2.7. Performance Issues in Networks

Network performance refers to the quality and speed of a network's transmission of data between devices. It is typically measured by factors such as bandwidth, latency, and throughput.

Network performance is important because it determines how well devices can communicate with each other and access the resources they need, such as the internet or shared files. Poor network performance can lead to slow response times, reduced productivity, and other problems.

Five Common Potential Issues that can Affect Network Performance

- **Bandwidth bottlenecks** − If the network's available bandwidth is inadequate for the number and type of devices and applications using it, performance can suffer.
- **Interference** − Physical objects or other electronic devices can interfere with wireless signals, causing them to degrade and reducing network performance.
- **Congestion** − When too many devices are trying to use the network at the same time, congestion can occur, leading to slow performance.
- **Malware** − Malware, such as viruses and worms, can compromise the performance of individual devices and the network as a whole.
- **Outdated hardware or software** − Using outdated equipment or software can limit the network's capabilities and lead to poor performance.

Bandwidth Bottlenecks

A bandwidth bottleneck is a network performance issue that occurs when the available bandwidth of the network is not sufficient to handle the volume of data being transmitted. This can result in slow response times and decreased performance for devices on the network.

Bandwidth is the maximum amount of data that can be transmitted over a network at any given time, and it is typically measured in bits per second (bps). If the demand for data transmission exceeds the available bandwidth, a bottleneck can occur. This can be caused by a variety of factors, such as an increase in the number of devices on the network, the use of bandwidth-intensive applications or services, or the presence of bottlenecks at specific points in the network.

To address a bandwidth bottleneck, you may need to upgrade your network's infrastructure or optimize your use of bandwidth by prioritizing certain types of traffic or limiting the use of

bandwidth-intensive applications. It may also be necessary to increase the available bandwidth by adding additional capacity to the network.

Interference

Interface-related network performance issues can occur when there are problems with the hardware or software interfaces that connect devices to the network. These issues can affect the ability of devices to communicate with each other and access network resources, leading to reduced performance.

Some common interface-related problems include −

- **Incorrectly configured interfaces** − If an interface is not configured properly, it may not be able to communicate with other devices on the network.
- **Faulty hardware** − Physical issues with an interface, such as a damaged connector or malfunctioning hardware, can prevent it from working properly.
- **Incompatible software** − If the software that controls an interface is not compatible with the rest of the network, it may not function correctly.

To troubleshoot interface-related network performance issues, you may need to check the configuration of the interfaces, replace faulty hardware, or update the software that controls the interfaces.

Congestion

Network congestion is a performance issue that occurs when there are too many devices trying to use the network at the same time. This can result in slow response times, dropped connections, and other problems.

There are several factors that can contribute to congestion on a network, including −

- An increase in the number of devices on the network − As more devices are added to a network, they compete for bandwidth and other resources, which can lead to congestion.

- The use of bandwidth-intensive applications − Applications that require a lot of bandwidth, such as streaming video or online gaming, can consume a large portion of the available bandwidth and cause congestion for other devices on the network.
- Limited bandwidth − If the available bandwidth of the network is insufficient to handle the volume of data being transmitted, congestion can occur.

To address congestion on a network, you may need to optimize your use of bandwidth by limiting the use of bandwidth-intensive applications or prioritizing certain types of traffic. You may also need to increase the available bandwidth by adding additional capacity to the network.

Malware

Malware is software that is specifically designed to harm or exploit computer systems. It can take many forms, including viruses, worms, Trojans, and ransomware. Malware can have a significant impact on network performance by consuming resources, slowing down devices, and disrupting communication between devices.

Some common ways in which malware can affect network performance include −

- **Consuming resources** − Malware can consume a large amount of a device's resources, such as CPU time and memory, which can lead to reduced performance.
- **Disrupting communication** − Malware can interfere with the normal operation of a device's network interface, preventing it from communicating with other devices on the network.
- **Spreading to other devices** − Malware can spread from one infected device to others on the network, potentially infecting multiple devices and causing widespread performance issues.

To address malware-related network performance issues, it is important to protect your devices with antivirus software and keep it up to date. You should also be careful about what you download and install, and avoid opening suspicious emails or visiting un trusted websites. If malware does infect your network, you may need to take steps to remove it, such as running antivirus software or restoring infected devices to a known good state.

Outdated Hardware or Software

Hardware and software issues can both affect network performance. Hardware problems can occur when there are physical issues with the devices or equipment that make up the network, while software problems can occur when there are issues with the programs and operating systems that run on the devices.

Some common hardware-related network performance issues include −

- **Faulty hardware** − If a device or piece of equipment is malfunctioning, it can disrupt communication on the network and reduce performance.
- **Incompatible hardware** − If the hardware on a device is not compatible with the rest of the network, it may not function correctly and could cause performance issues.
- **Insufficient hardware resources** − If a device does not have enough processing power, memory, or other resources, it may not be able to handle the demands placed on it, leading to reduced performance.

Some common software-related network performance issues include −

- **Outdated software** − If the software on a device is outdated, it may not be able to take advantage of newer technologies or may not be compatible with the rest of the network, leading to reduced performance.
- **Software bugs** − If the software on a device contains bugs or errors, it can cause performance issues or even crash the device.
- **Inefficient software** − If the software on a device is not optimized for performance, it may consume more resources than necessary, leading to reduced performance.

To address hardware and software-related network performance issues, you may need to upgrade or replace outdated or faulty equipment, update software to the latest version, or optimize the software on your devices to improve performance.

2.8. Delay Tolerant Networking

Delay-tolerant networking (**DTN**) is an approach to computer network architecture that seeks to address the technical issues in heterogeneous networks that may lack continuous network connectivity. Examples of such networks are those operating in mobile or extreme terrestrial environments, or planned networks in space.

Recently, the term **disruption-tolerant networking** has gained currency in the United States due to support from DARPA, which has funded many DTN projects. Disruption may occur because of the limits of wireless radio range, sparsity of mobile nodes, energy resources, attack, and noise.

Routing

The ability to transport, or route, data from a source to a destination is a fundamental ability all communication networks must have. Delay and disruption-tolerant networks (DTNs), are characterized by their lack of connectivity, resulting in a lack of instantaneous end-to-end paths. In these challenging environments, popular ad hoc routing protocols such as AODV and DSR fail to establish routes. This is due to these protocols trying to first establish a complete route and then, after the route has been established, forward the actual data. However, when instantaneous end-to-end paths are difficult or impossible to establish, routing protocols must take to a "store and forward" approach, where data is incrementally moved and stored throughout the network in hopes that it will eventually reach its destination. A common technique used to maximize the probability of a message being successfully transferred is to replicate many copies of the message in the hope that one will succeed in reaching its destination. This is feasible only on networks with large amounts of local storage and internode bandwidth relative to the expected traffic. In many common problem spaces, this inefficiency is outweighed by the increased efficiency and shortened delivery times made possible by taking maximum advantage of available unscheduled forwarding opportunities. In others, where available storage and internode throughput opportunities are more tightly constrained, a more discriminate algorithm is required.

Bundle protocols

Bundles are routed in a store and forward manner between participating nodes over varied network transport technologies (including both IP and non-IP based transports). The transport

layers carrying the bundles across their local networks are called *bundle convergence layers.* The bundle architecture therefore operates as an overlay network, providing a new naming architecture based on Endpoint Identifiers (EIDs) and coarse-grained class of service offerings.

Protocols using bundling must leverage application-level preferences for sending bundles across a network. Due to the store and forward nature of delay-tolerant protocols, routing solutions for delay-tolerant networks can benefit from exposure to application-layer information. For example, network scheduling can be influenced if application data must be received in its entirety, quickly, or without variation in packet delay. Bundle protocols collect application data into bundles that can be sent across heterogeneous network configurations with high-level service guarantees. The service guarantees are generally set by the application level, and the RFC 5050 Bundle Protocol specification includes "bulk", "normal", and "expedited" markings.

Security issues

Security concerns for delay-tolerant networks vary depending on the environment and application, though authentication and privacy are often critical. These security guarantees are difficult to establish in a network without continuous bi-directional end-to-end paths between devices because the network hinders complicated cryptographic protocols, hinders key exchange, and each device must identify other intermittently visible devices. Solutions have typically been modified from mobile ad hoc network and distributed security research, such as the use of distributed certificate authorities[and PKI schemes. Original solutions from the delay-tolerant research community include:

1) the use of identity-based encryption, which allows nodes to receive information encrypted with their public identifier

2) the use of tamper-evident tables with a gossiping protocol;